

Problem Formulation

The SABL library is designed to solve constrained optimization problems of the form

$$\begin{aligned} \min_{u,z} J(u, z) \\ \text{s.t.} \quad c(u, z) = 0 \end{aligned} \quad (1)$$

via a reduced space optimization strategy. Here, u indicates the state variable, z is the control variable, J is the objective function, and c specifies the constraints. If $c(u, z) = 0$ admits a unique solution for any given z , then there exists a solution operator $S(z)$ such that $c(S(z), z) = 0 \forall z$. We may reformulate (1) in reduced space and solve the equivalent optimization problem

$$\min_z \hat{J}(z) = J(S(z), z), \quad (2)$$

which will be the focus of this document and the corresponding MATLAB codes. Along with solving (2), we would also like the code to facilitate the use of hyper-differential sensitivity analysis (HDSA) for the solution of (2).

Note that (1) and (2) may be posed as infinite-dimensional optimizations over function spaces. After discretization for computation, we have finite-dimensional $u \in \mathbb{R}^{n_u}$ and $z \in \mathbb{R}^{n_z}$. With these dimensions, we have $J : \mathbb{R}^{n_u} \times \mathbb{R}^{n_z} \rightarrow \mathbb{R}$, $c : \mathbb{R}^{n_u} \times \mathbb{R}^{n_z} \rightarrow \mathbb{R}^{n_c}$, $S : \mathbb{R}^{n_z} \rightarrow \mathbb{R}^{n_u}$, and $\hat{J} : \mathbb{R}^{n_z} \rightarrow \mathbb{R}$, where n_c is the number of equations needed to express the constraints.

Base Class: Constrained_Optimization

We adopt an object-oriented design in MATLAB by defining a base class `Constrained_Optimization` where the problem, optimization, and HDSA interfaces will be defined. To facilitate our analysis, we require efficient algorithms to compute the gradient $\nabla_z \hat{J}(z) \in \mathbb{R}^{n_z}$ and Hessian-vector products $\nabla_{z,z} \hat{J}(z)v$ for a given vector $v \in \mathbb{R}^{n_z}$ without explicitly forming the Hessian $\nabla_{z,z} \hat{J}(z) \in \mathbb{R}^{n_z \times n_z}$. To this end we introduce adjoint-based derivatives which is implemented in the code. Algorithm 1 demonstrates how to efficiently calculate the gradient. Throughout this document we use $c_u(u, z) \in \mathbb{R}^{n_c \times n_u}$ and $c_z(u, z) \in \mathbb{R}^{n_c \times n_z}$ to denote the Jacobians of c with respect to u and z , respectively, $\nabla_u J(u, z) \in \mathbb{R}^{n_u}$ and $\nabla_z J(u, z) \in \mathbb{R}^{n_z}$ to denote the gradients (where u and z are independent of one another) of J with respect to u and z , respectively, and a superscript T to denote a matrix transpose. Based on Algorithm 1, Table 1 lists the pure virtual functions defined in `Constrained_Optimization` which must be implemented for a given problem to enable the gradient computation.

Algorithm 1 Adjoint-based gradient calculation

Input: $\bar{z} \in \mathbb{R}^{n_z}$

- 1: Solve the state equation to determine $\bar{u} \in \mathbb{R}^{n_u}$ such that $c(\bar{u}, \bar{z}) = 0$
- 2: Solve the adjoint equation $c_u(\bar{u}, \bar{z})^{\mathsf{T}} \bar{\lambda} = -\nabla_u J(\bar{u}, \bar{z})$ for $\bar{\lambda} \in \mathbb{R}^{n_c}$ (a linear solve)
- 3: Compute $\nabla_z \hat{J}(\bar{z}) = c_z(\bar{u}, \bar{z})^{\mathsf{T}} \bar{\lambda} + \nabla_z J(\bar{u}, \bar{z})$

Return: $\nabla_z \hat{J}(\bar{z}) \in \mathbb{R}^{n_z}$

Function Signature	Mathematical Description
<code>[val, grad_u, grad_z] = Objective(u, z)</code>	Evaluate J , $\nabla_u J$, and $\nabla_z J$
<code>[u] = State_Solve(z)</code>	Solve $c(u, z) = 0$ for u
<code>[Mv] = c_u.Transpose_Inverse_Apply(v, u, z)</code>	Compute the product $c_u(u, z)^{-\mathsf{T}} v$
<code>[Mv] = c_z.Transpose_Apply(v, u, z)</code>	Compute the product $c_z(u, z)^{\mathsf{T}} v$

Table 1: Pure virtual function in `Constrained_Optimization` needed to execute Algorithm 1.

To compute efficient Hessian-vector products, Algorithm 2 outlines computation using incremental state and incremental adjoint equations. We use $\lambda^{\mathsf{T}} c_{u,u}(u, z) \in \mathbb{R}^{n_u \times n_u}$ and $\nabla_{u,u} J(u, z) \in \mathbb{R}^{n_u \times n_u}$ to

denote the (u, u) Hessian of the scalar functions $\lambda^\top c$ and J , respectively, and similar expressions for the (u, z) , (z, u) , and (z, z) Hessians. Table 2 lists the pure virtual functions in **Constrained_Optimization** which must be implemented for a given problem to enable Hessian-vector product computations using Algorithm 2.

Algorithm 2 Adjoint-based Hessian-vector product calculation

Input: $v \in \mathbb{R}^{n_z}$, $\bar{u} \in \mathbb{R}^{n_u}$, $\bar{z} \in \mathbb{R}^{n_z}$, $\bar{\lambda} \in \mathbb{R}^{n_c}$

- 1: Compute $w = c_z(\bar{u}, \bar{z})v \in \mathbb{R}^{n_c}$
 - 2: Solve the incremental state equation $c_u(\bar{u}, \bar{z})\bar{\mu} = -w$ for $\bar{\mu} \in \mathbb{R}^{n_u}$ (a linear solve)
 - 3: Compute $y_J = \nabla_{u,u}J(\bar{u}, \bar{z})\bar{\mu} + \nabla_{u,z}J(\bar{u}, \bar{z})v \in \mathbb{R}^{n_u}$
 - 4: Compute $y_c = \bar{\lambda}^\top c_{u,u}(\bar{u}, \bar{z})\bar{\mu} + \bar{\lambda}^\top c_{u,z}(\bar{u}, \bar{z})v \in \mathbb{R}^{n_u}$
 - 5: Solve the incremental adjoint equation $c_u(\bar{u}, \bar{z})^\top \bar{\gamma} = -(y_J + y_c)$ for $\bar{\gamma} \in \mathbb{R}^{n_c}$ (a linear solve)
 - 6: Compute $x_J = \nabla_{z,u}J(\bar{u}, \bar{z})\bar{\mu} + \nabla_{z,z}J(\bar{u}, \bar{z})v \in \mathbb{R}^{n_z}$
 - 7: Compute $x_c = c_z(\bar{u}, \bar{z})^\top \bar{\gamma} + \bar{\lambda}^\top c_{z,u}(\bar{u}, \bar{z})\bar{\mu} + \bar{\lambda}^\top c_{z,z}(\bar{u}, \bar{z})v \in \mathbb{R}^{n_z}$
 - 8: Compute $\nabla_{z,z}\hat{J}(\bar{z})v = x_J + x_c$
- Return:** $\nabla_{z,z}\hat{J}(\bar{z})v \in \mathbb{R}^{n_z}$
-

Function Signature	Mathematical Description
[Mv] = c_u_Inverse_Apply(v,u,z)	Compute the product $c_u(u, z)^{-1}v$
[Mv] = c_z_Apply(v,u,z)	Compute the product $c_z(u, z)v$
[Mv] = c_uu_Apply(v,u,z,lambda)	Compute the product $\lambda^\top c_{u,u}(u, z)v$
[Mv] = c_uz_Apply(v,u,z,lambda)	Compute the product $\lambda^\top c_{u,z}(u, z)v$
[Mv] = c_zu_Apply(v,u,z,lambda)	Compute the product $\lambda^\top c_{z,u}(u, z)v$
[Mv] = c_zz_Apply(v,u,z,lambda)	Compute the product $\lambda^\top c_{z,z}(u, z)v$
[Mv] = Objective_uu_Apply(v,u,z)	Compute the product $\nabla_{u,u}J(u, z)v$
[Mv] = Objective_uz_Apply(v,u,z)	Compute the product $\nabla_{u,z}J(u, z)v$
[Mv] = Objective_zu_Apply(v,u,z)	Compute the product $\nabla_{z,u}J(u, z)v$
[Mv] = Objective_zz_Apply(v,u,z)	Compute the product $\nabla_{z,z}J(u, z)v$

Table 2: Pure virtual function in **Constrained_Optimization** needed to execute Algorithm 2.

We implement Algorithm 1 and Algorithm 2 in **Constrained_Optimization**'s private functions **Jhat()** and **Jhat_hessVec()**, respectively, and interface them with MATLAB's solver **fminunc()** in the public function **Optimize()**. In addition, **Constrained_Optimization** has the public functions **Finite_Difference_Gradient_Check()** and **Finite_Difference_Hessian_Check()** to test the accuracy of our calculation of $\nabla_z \hat{J}(z)$ and $\nabla_{z,z} \hat{J}(z)v$.

Derived Class: Constrained_ODE_Optimization

We specialize the general optimization problem (1) and consider the case where $c(u, z) = 0$ corresponds to the discretization of a system of ordinary differential equations (ODEs)

$$\begin{aligned} \frac{dy}{dt} &= f(y, z, t) \\ y(0) &= h(z) \end{aligned}$$

where $y : [0, T] \rightarrow \mathbb{R}^m$, $T > 0$, $z \in \mathbb{R}^n$, $f : \mathbb{R}^m \times \mathbb{R}^n \times [0, T] \rightarrow \mathbb{R}^m$, $h : \mathbb{R}^n \rightarrow \mathbb{R}^m$, and u collects $y(t)$ at all monitored times t (we make this precise shortly). The objective function is

$$J(u, z) = \int_0^T g(y(t), t)dt + R(z)$$

where $g : \mathbb{R}^m \times \mathbb{R} \rightarrow \mathbb{R}$ and $R : \mathbb{R}^n \rightarrow \mathbb{R}$ are user-specified functions. This scenario is encapsulated in the subclass `Constrained_ODE_Optimization`, derived from the base class `Constrained_Optimization`. The subclass inputs the state dimension m , the final time T , and the number of time steps N , and requires the user to specify the objective function and ODE system through the pure virtual functions summarized in Table 3.

Function Signature	Mathematical Description
<code>[val,grad_y] = Time_Instance_Objective(y,t)</code>	Evaluate $g(y, t)$ and $\nabla_y g(y, t)$
<code>[val,grad_z] = Regularization_Objective(z)</code>	Evaluate $R(z)$ and $\nabla_z R(z)$
<code>[f,f_y,f_z] = Time_Instance_RHS(y,z,t)</code>	Evaluate $f(y, z, t)$, $f_y(y, z, t)$, and $f_z(y, z, t)$
<code>[h, h_z] = Initial_Condition(z)</code>	Evaluate $h(z)$ and $h_z(z)$
<code>[Mv] = Time_Instance_Objective_yy_Apply(v,y,t)</code>	Compute the product $\nabla_{y,y} g(y, t)v$
<code>[Mv] = Regularization_Objective_zz_Apply(v,z)</code>	Compute the product $\nabla_{z,z} R(z)v$
<code>[Mv] = Time_Instance_RHS_yy_Apply(v,y,z,t,lambda)</code>	Compute the product $\lambda^T f_{y,y}(y, z, t)v$
<code>[Mv] = Time_Instance_RHS_yz_Apply(v,y,z,t,lambda)</code>	Compute the product $\lambda^T f_{y,z}(y, z, t)v$
<code>[Mv] = Time_Instance_RHS_zy_Apply(v,y,z,t,lambda)</code>	Compute the product $\lambda^T f_{z,y}(y, z, t)v$
<code>[Mv] = Time_Instance_RHS_zz_Apply(v,y,z,t,lambda)</code>	Compute the product $\lambda^T f_{z,z}(y, z, t)v$
<code>[Mv] = Initial_Condition_zz_Apply(v,z,lambda)</code>	Compute the product $\lambda^T h_{z,z}(z)v$

Table 3: Pure virtual function in `Constrained_ODE_Optimization`.

In order to implement the pure virtual functions of `Constrained_Optimization` (the functions listed in Table 1 and Table 2), we must fix a time discretization scheme and write a finite dimensional optimization problem in the form of (1). To this end, let $t_k = T \frac{k-1}{N-1}$, $k = 1, 2, \dots, N$ be equally spaced nodes in the time interval $[0, T]$ and $y_{i,k}$ denote the value of the i^{th} component of $y(t_k) \in \mathbb{R}^m$, i.e. the i^{th} state variable (of m variables) at the k^{th} time (of N nodes). Concatenating these discrete evaluations yields the space-time state vector

$$u = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^{mN}, \quad \text{where} \quad y_k = \begin{pmatrix} y_{1,k} \\ y_{2,k} \\ \vdots \\ y_{m,k} \end{pmatrix} \in \mathbb{R}^m.$$

The full state $u \in \mathbb{R}^{mN}$ therefore consists of the values of the discrete ODE state $y(t) \in \mathbb{R}^m$ at all N times $t_1, \dots, t_N$. Thus, in this case we have $n_u = mN$ and $n_z = n$. Next, applying the backward Euler time integration scheme, we have

$$c(u, z) = \begin{pmatrix} y_1 - h(z) \\ y_2 - y_1 - \Delta t f(y_2, z, t_2) \\ y_3 - y_2 - \Delta t f(y_3, z, t_3) \\ y_4 - y_3 - \Delta t f(y_4, z, t_4) \\ \vdots \\ y_N - y_{N-1} - \Delta t f(y_N, z, t_N) \end{pmatrix} \in \mathbb{R}^{mN}, \quad (3)$$

where $\Delta t = \frac{T}{N-1}$. Hence, $n_c = n_u = mN$. Finally, applying the trapezoid rule for integration, the discretized objective function is

$$J(u, z) = \sum_{k=1}^N w_k g(y_k, t_k) + R(z), \quad (4)$$

where $w_k = \Delta T$, $k = 2, 3, \dots, N-1$, and $w_1 = w_N = 0.5\Delta T$, are integration weights.

Given an implementation of g , R , and their derivatives, the pure virtual functions corresponding to the objective J and its derivatives are easily implemented based on (4). The pure virtual functions

corresponding to the constraint c require more careful consideration. To solve the equation $c(u, z) = 0$, we exploit the block structure in (3) and solve $m \times m$ systems of equations at each of the N time points sequentially (i.e., perform time integration rather than solving $c(u, z) = 0$ as a system of mN coupled equations). However, the structure of $c(u, z)$ is critical for ensuring that the adjoints are computed correctly. The Jacobian of c with respect to u , $c_u(u, z)$, is the $mN \times mN$ matrix

$$\begin{pmatrix} I_m & 0 & 0 & 0 & \dots & 0 \\ -I_m & I_m - \Delta t f_y(y_2, z, t_2) & 0 & 0 & \dots & 0 \\ 0 & -I_m & I_m - \Delta t f_y(y_3, z, t_3) & 0 & \dots & 0 \\ 0 & 0 & -I_m & I_m - \Delta t f_y(y_4, z, t_4) & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 & -I_m & I_m - \Delta t f_y(y_N, z, t_N) \end{pmatrix}.$$

Hence the implementation of $c_u(u, z)^{-\top} v$ for the adjoint solve in Algorithm 1 must consider the system of linear equations $c_u(u, z)^{\top} x = v$ written as

$$\begin{pmatrix} I_m & -I_m & 0 & 0 & \dots & 0 \\ 0 & I_m - \Delta t f_y(y_2, z, t_2) & -I_m & 0 & \dots & 0 \\ 0 & 0 & I_m - \Delta t f_y(y_3, z, t_3) & -I_m & \dots & 0 \\ 0 & 0 & 0 & I_m - \Delta t f_y(y_4, z, t_4) & -I_m & \dots \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 & 0 & I_m - \Delta t f_y(y_N, z, t_N) \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_N \end{pmatrix} = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ \vdots \\ v_N \end{pmatrix}$$

and compute $x = c_u(u, z)^{-\top} v$ via the backward-in-time stepping algorithm

$$\begin{aligned} x_N &= (I_m - \Delta t f_y(y_N, z, t_N))^{-1} v_N \\ x_k &= (I_m - \Delta t f_y(y_k, z, t_k))^{-1} (v_k + x_{k+1}) \quad k = N-1, N-2, \dots, 2 \\ x_1 &= v_1 + x_2. \end{aligned}$$

This ensures that the time discretization of the adjoint equation is consistent with the time integration of the state equation at the discrete level, thus providing an accurate gradient computation. Similar care must be taken when implementing all other matrix-vector products and linear solves involving derivatives of the constraint c .